



Figure 1: A typical R studio window

A sample output follows:

```
[1] 11 14 17 20 23 26 29
```

In Figure 1, the command *from* is for specifying the starting value of the sequence, *to* is for specifying the ending value of the sequence, and *by* is to increment the sequence.

The concatenation function can be used to store values in *x1*, as shown below:

```
> x1 <- c(11,12,13,14,15,16,17,18,19,20)
#assigning integers in the range
#11 to 20
> x1
#print the values stored in x1
```

Random numbers in R

Random numbers are used in simulation and they are also used by statisticians.

To generate a random number (having a fractional part) between 11 and 15, and store it in *rnum*, the following command can be used:

```
> rnum <- runif(1,11,15)
> rnum # print the value of rnum
[1] 14.75596 (Sample output)
```

By default, *runif(n)* generates random numbers in the range of 0 to 1.

```
> rnum <- runif(10,11,15) # for generating 10 random numbers
# in the range of 11 to 15
```

In order to generate three integer random numbers between 11 and 15, the following command can be used:

```
> x <- sample(11:15,3,replace=T)
> x
```

11:15 is the range between which random numbers have to be generated, 3 the number of random numbers to be generated and *replace = T*, which indicates that repeats are permissible.

For generating a sequence of random numbers, and to generate the same sequence later, *set.seed()* and *runif()* can be used as follows:

```
> set.seed(7)
> runif(7)
```



Note: 1) Help is readily available in R. In order to obtain help on random numbers, the following command might be helpful:

```
help.search("random numbers")
```

- 2) *<-* (Less than, followed by the minus symbol) is the assignment operator in R. Alternatively, the *'='* operator can also be used.
- 3) R is a case-sensitive language.

The section below dwells on vector and matrix operations in R.

Vectors in R

The R programming environment provides very powerful vector and matrix operation tools. The code below creates a vector for storing the first 13 members of the Fibonacci series:

```
Code snippet 1
> fib <- numeric(13)

> fib[1] <- 0
> fib[2] <- 1
> for(idx in 3:13){
+   fib[idx] <- fib[idx-1] + fib[idx - 2]
+ }
> fib
```

The output of the above code would be:

```
[output] 0 1 1 2 3 5 8 13 21 34 55 89
144
```

- *fib <- numeric(13)*: Sets up a numeric vector the length of which is 13 and this vector is initialised with zeroes.
- *fib[1] <- 0*: Sets the first element of vector *fib* to the value 0. The next statement is self-explanatory.
- The *for()* loop computes the third to the thirteenth element (except for the first two elements, each term in the Fibonacci series is the sum of the preceding two terms).
- *fib*: Displays the elements of the vector *fib*.